

# **LAPORAN PRAKTIKUM INTERNET OF THINGS (IoT) II**

## **Modul 3**



Disusun Oleh :

Nama : Syarifah Febriyani

NPM : 25782085

Prodi : TRI C

Dosen Pengampu :

Dr. Septafiansyah Dwi Putra, ST, MT

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET  
JURUSAN TEKNOLOGI INFORMASI  
POLITEKNIK NEGERI LAMPUNG  
2025/2026**

# Jaringan Nirkabel dan Komunikasi HTTP

## Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

Blinkino

```
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Redmi";
6  const char* password = "123456789";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
19   <title>IoT Dashboard</title>
20   <style>
21     body { font-family: Arial; text-align: center; margin-top: 50px; }
22     button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

New Line 115200 baud

IP Address Server Anda:  
10.207.103.114  
.....  
IP Address Server Anda:  
10.207.103.114



 10.207.103.114



 29



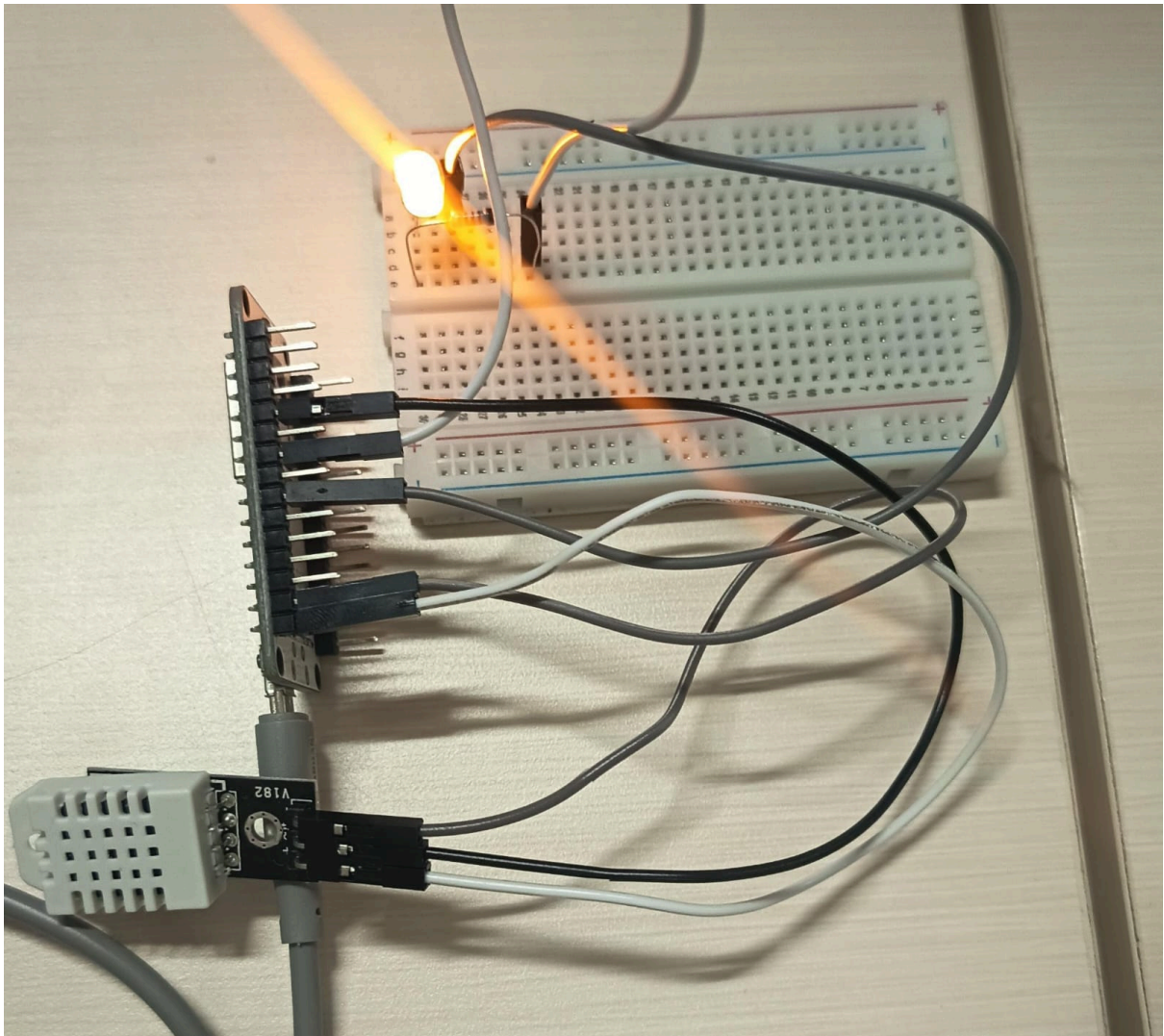
## ESP8266 Web Server

Suhu Saat Ini: **31.90** Celcius

### Kendali Relay

ON

OFF



Setelah program berhasil di-upload, NodeMCU dapat terhubung ke Wi-Fi dan mendapatkan IP Address. Halaman Web Server bisa dibuka melalui browser. Suhu dari sensor DHT22 tampil di halaman web, dan tombol ON/OFF dapat digunakan untuk mengontrol LED.

## Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

### Server

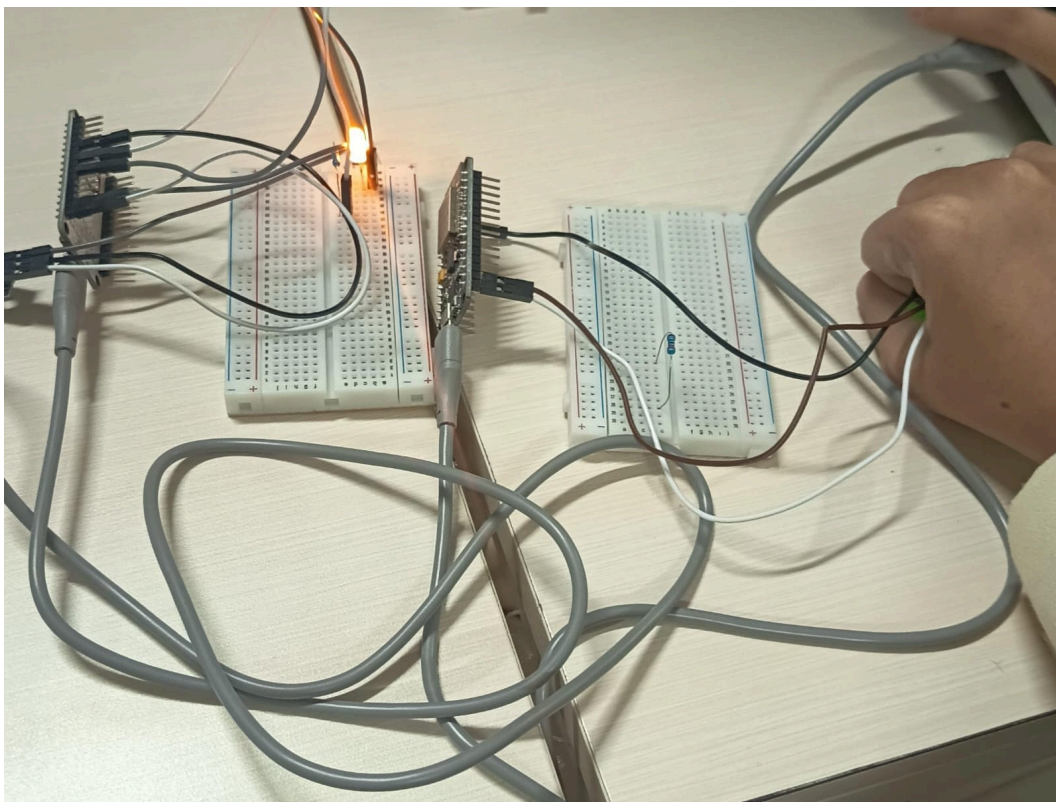
```
Blinkino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "sofy";
6  const char* password = "12345678";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
19   <title>IoT Dashboard</title>
20   <style>
21     body { font-family: Arial; text-align: center; margin-top: 50px; }
22     button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

New Line 115200 bau

Client Terhubung ke Wi-Fi!  
HTTP Response code: -1  
.....  
IP Address Server Anda:  
10.235.57.114



## ESP8266 Web Server

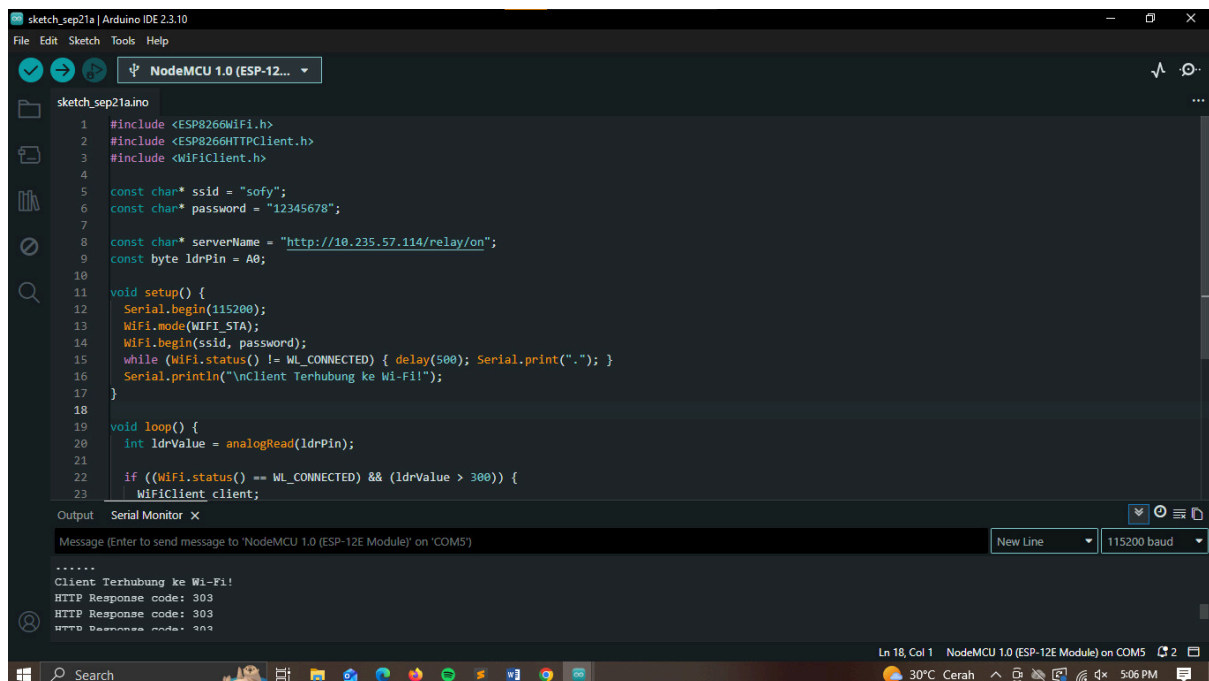
Suhu Saat Ini: **29.10** Celcius

Kendali Relay

ON

OFF

client



```
sketch_sep21a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "sofy";
6 const char* password = "12345678";
7
8 const char* serverName = "http://10.235.57.114/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int ldrValue = analogRead(ldrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23     WiFiClient client;
24     HTTPClient http;
25     http.begin(client, serverName);
26     http.GET();
27     Serial.println("HTTP Response code: " + http.getString());
28     http.end();
29   }
30 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

.....

Client Terhubung ke Wi-Fi!

HTTP Response code: 303

HTTP Response code: 303

Ln 18, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM5 2 115200 baud

Saat LDR ditutup dan nilai pembacaan sensor kurang dari 300, NodeMCU Client mengirim HTTP GET ke Server. Jika koneksi berhasil, Server menerima perintah dan LED menyala. Serial Monitor menampilkan kode respons HTTP, misalnya 303 ketika Server mengarahkan kembali browser ke halaman utama.

### Latihan 3.6

- Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

jawab :

Mode jaringan Station (STA) digunakan pada kedua praktikum karena NodeMCU perlu terhubung ke jaringan Wi-Fi yang sudah tersedia. Pada Praktikum 1, NodeMCU berfungsi sebagai Web Server sehingga harus terhubung ke Wi-Fi agar HP atau laptop pada jaringan yang sama dapat mengakses alamat IP Web Server melalui browser. Pada Praktikum 2, NodeMCU berfungsi sebagai HTTP Client yang harus berkomunikasi dengan NodeMCU Server milik teman melalui jaringan Wi-Fi yang sama.

Mode Access Point (AP) tidak digunakan karena pada mode ini NodeMCU membuat jaringan Wi-Fi sendiri. Sementara itu, skenario praktikum membutuhkan beberapa perangkat terhubung pada jaringan lokal Wi-Fi yang sama agar dapat saling berkomunikasi. Oleh karena itu, mode Station (STA) lebih sesuai dengan kebutuhan kedua praktikum.

- Latihan 2 (Modifikasi Antarmuka & Telemetry): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*

jawab :

i. Untuk menampilkan kelembapan, kita menambahkan pembacaan menggunakan fungsi `dht.readHumidity()`. Kemudian dibuat placeholder `%HUMIDITY%` pada HTML dan diganti dengan nilai kelembapan di fungsi `handleRoot()`.

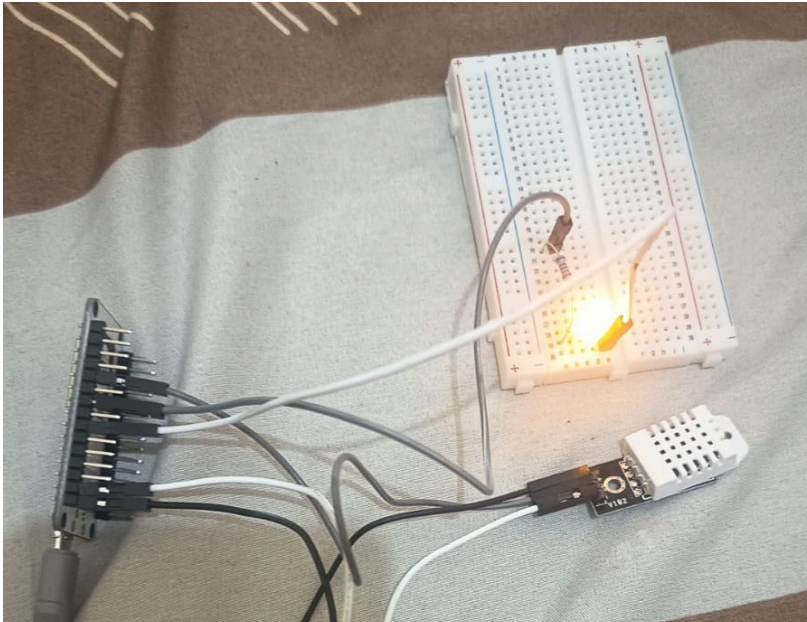
## ESP8266 Web Server

Suhu Saat Ini: **31.10** Celcius

Kelembapan: **74.70** %

Kendali Relay





ii. Modifikasi ini bertujuan agar tombol ON/OFF menjadi satu tombol saja. Tulisan tombol berubah sesuai kondisi LED. Kalau LED mati, tombol menampilkan "NYALAKAN (ON)". Kalau LED menyala, tombol berubah menjadi "MATIKAN (OFF)"

## ESP8266 Web Server

Suhu Saat Ini: **31.00** Celcius

Kelembapan: **75.10** %

Kendali Relay

NYALAKAN (ON)

- Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

```
sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "sofy";
6 const char* password = "12345678";
7
8 const char* serverName = "http://10.235.57.114/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int ldrValue = analogRead(ldrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23     WiFiClient client;
```

Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

HTTP Response code: 303  
HTTP Response code: -1  
HTTP Response code: -1  
HTTP Response code: -1

Ln 18, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM5 115200 baud

Jika NodeMCU Server kehilangan daya, Client tidak bisa terhubung ke alamat IP Server. Akibatnya, `http.GET()` akan menghasilkan kode error negatif, bukan kode sukses seperti 200 atau 303.

- Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

10.235.57.114

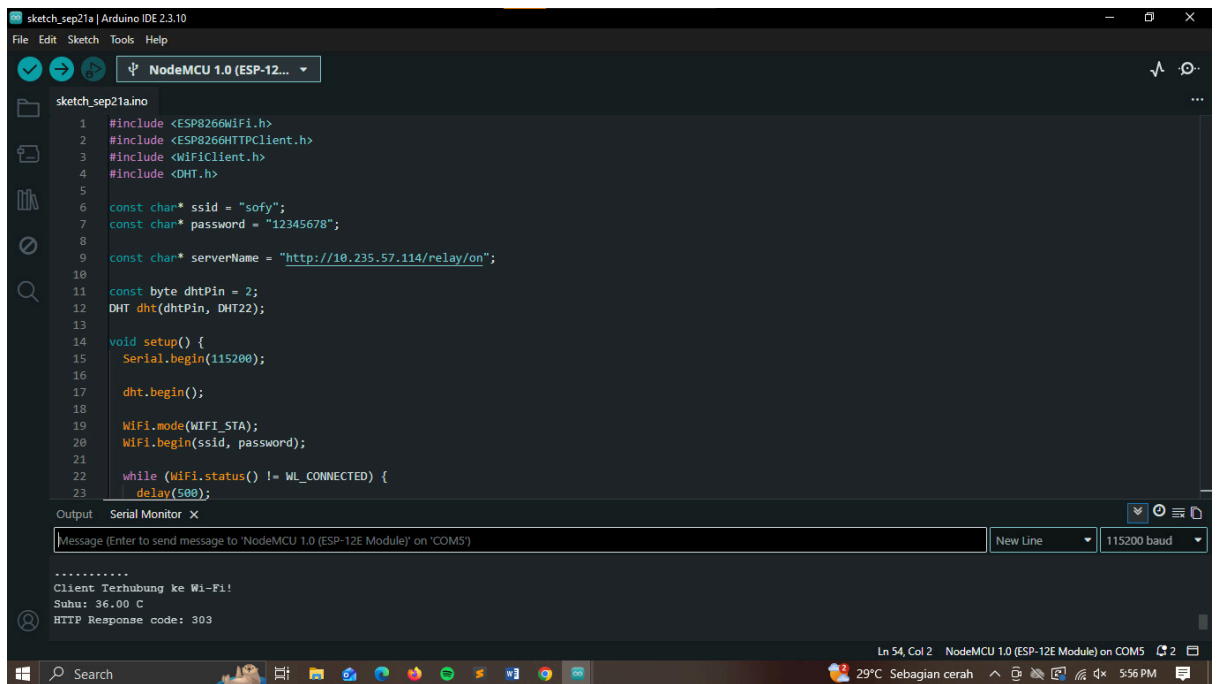
## ESP8266 Web Server

Suhu Saat Ini: **36.50** Celcius

Kendali Relay

ON

OFF

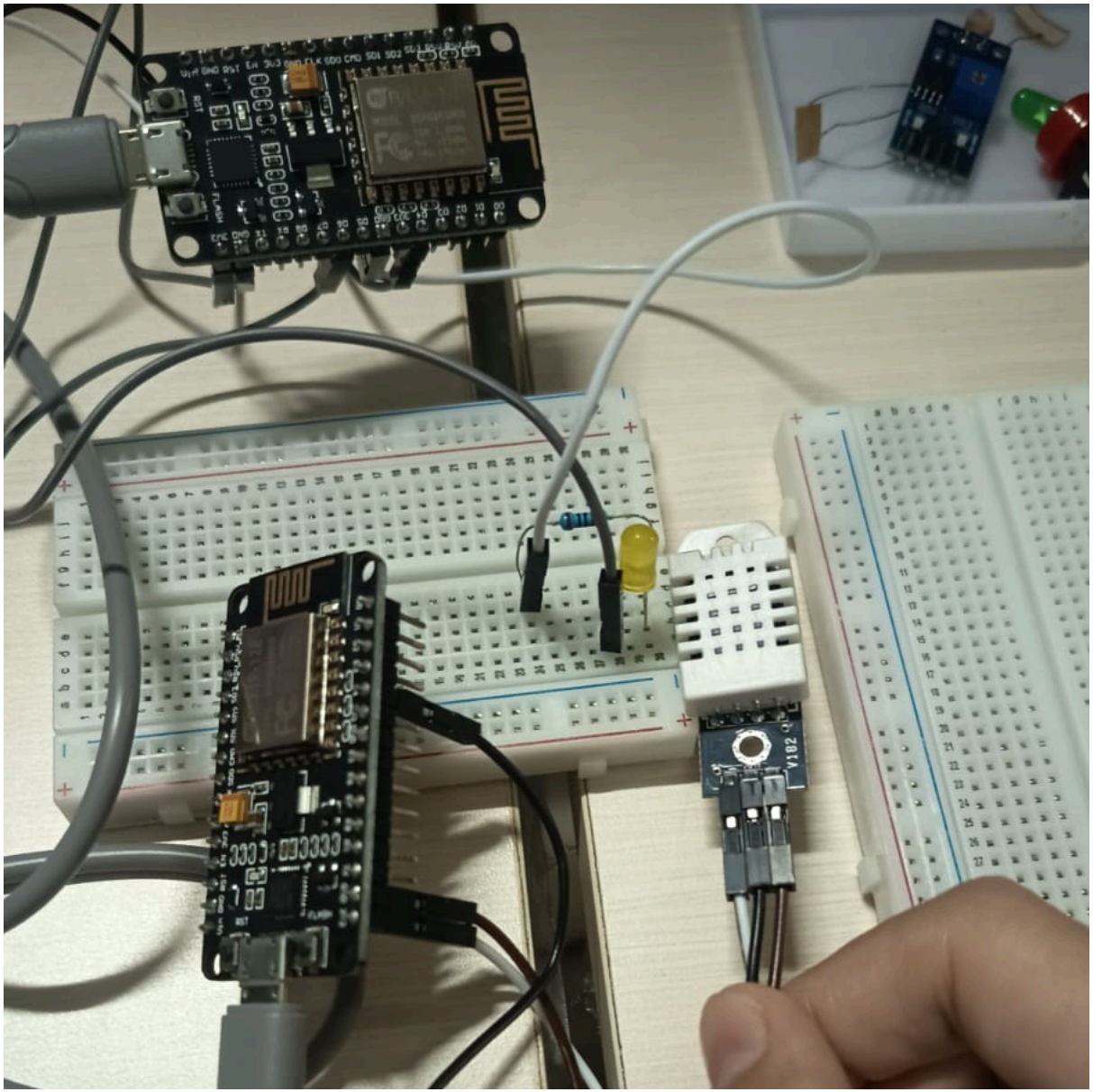


Client akan membaca suhu dari sensor DHT22. Jika suhu lebih dari 35°C dan Wi-Fi tersambung, Client mengirim HTTP GET ke Server. Kalau suhu 35°C atau kurang, Client tidak mengirim perintah.

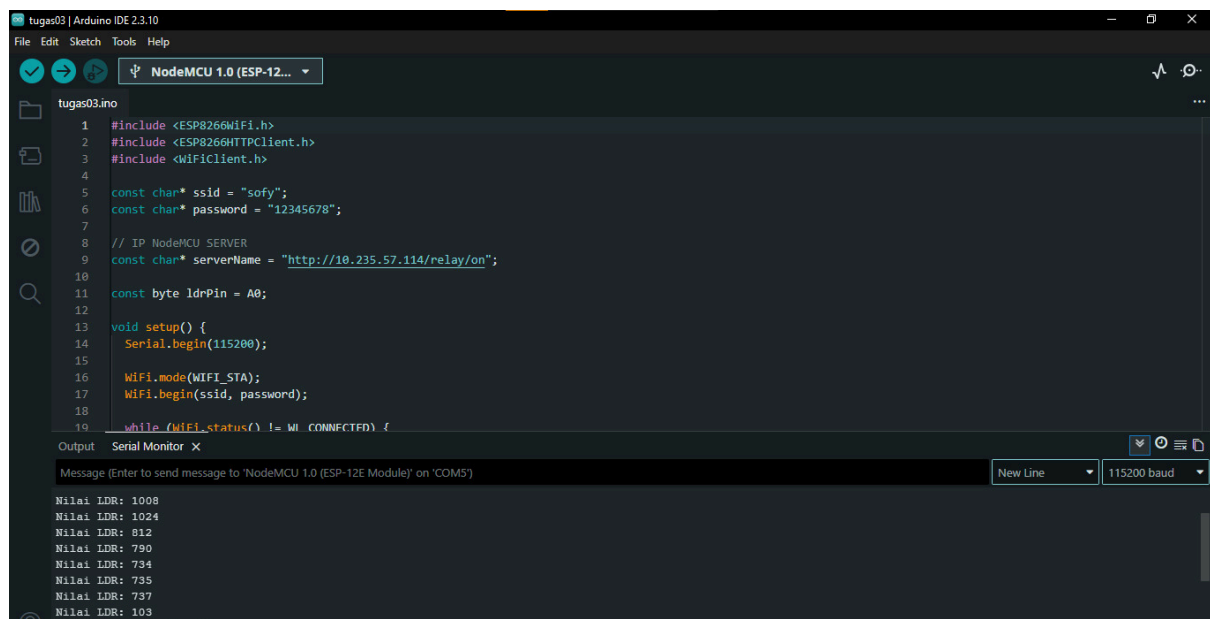
### Tugas 3.7 : Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

#### Server





client



```
tugas03.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "sofy";
6 const char* password = "12345678";
7
8 // IP NodeMCU SERVER
9 const char* serverName = "http://10.235.57.114/relay/on";
10
11 const byte ldrPin = A0;
12
13 void setup() {
14   Serial.begin(115200);
15
16   WiFi.mode(WIFI_STA);
17   WiFi.begin(ssid, password);
18
19   while (WiFi.status() != WL_CONNECTED) {
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Nilai LDR: 1008  
Nilai LDR: 1024  
Nilai LDR: 812  
Nilai LDR: 790  
Nilai LDR: 734  
Nilai LDR: 735  
Nilai LDR: 737  
Nilai LDR: 103

Tidak aman 10.235.57.114

## ESP8266 Web Server

Suhu Saat Ini: **29.10 °C**

Kelembapan: **45.60 %**

Kendali Relay



Halaman web akan memuat ulang secara otomatis setiap 5 detik. Jika suhu lingkungan berubah, nilai suhu yang ditampilkan juga akan ikut berubah setelah halaman dimuat ulang.

Menurut saya, Meta Refresh masih kurang cocok untuk dashboard yang benar-benar real-time. Halaman harus dimuat ulang secara keseluruhan setiap 5 detik, sehingga bisa terjadi kedipan layar atau flickering. Selain itu, browser harus kembali mengambil data HTML dari Server secara berulang, sehingga penggunaan bandwidth menjadi lebih besar dibandingkan jika hanya mengambil data sensor yang berubah. Tombol ON/OFF juga bisa terasa kurang nyaman digunakan. Misalnya, saat pengguna menekan tombol tepat ketika halaman sedang melakukan refresh, interaksi bisa terasa terganggu karena halaman sedang dimuat ulang.